Robot Esquiva-Obstáculos

```
#include <Servo.h>

// Definición de pines para el sensor ultrasónico HC-SR04
const int trigPin = 12;
const int echoPin = 13;

// Pin y objeto para el servomotor
const int servoPin = 10;
Servo myServo;

// Definición de pines para el Driver L298N y los Motores
const int enA = 5;  // Velocidad Motor Izquierdo
const int in1 = 7;  // Dirección Motor Izquierdo 1
const int in2 = 8;  // Dirección Motor Izquierdo 2
const int in3 = 9;  // Dirección Motor Derecho 1
const int in4 = 4;  // Dirección Motor Derecho 2
const int enB = 6;  // Velocidad Motor Derecho

void setup() {
  // Configurar pines del sensor ultrasónico
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  // Inicializar servomotor y centrarlo (90 grados)
  myServo.attach(servoPin);
  myServo.write(90);

  // Configurar pines de motores como salida
  pinMode(enA, OUTPUT);
  pinMode(enB, OUTPUT);
```

```
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);

  // Velocidad de crucero moderada para los motores (0 a 255)
  analogWrite(enA, 180);
  analogWrite(enB, 180);

  Serial.begin(9600);
}

void loop() {
  // Medir la distancia frontal
  int distance = medirDistancia();

  Serial.print("Distancia frontal: ");
  Serial.print(distance);
  Serial.println(" cm");

  // Si hay camino libre (más de 30 cm)
  if (distance > 30) {
    avanzar();
  }
  else {
    // Obstáculo detectado: Detener el robot
    detener();
    delay(300);

    // Retroceder un poco para tomar espacio
    retroceder();
```

```
    delay(400);
    detener();

    // Mirar hacia la izquierda (180 grados)
    myServo.write(180);
    delay(500);
    int distanceLeft = medirDistancia();

    // Mirar hacia la derecha (0 grados)
    myServo.write(0);
    delay(500);
    int distanceRight = medirDistancia();

    // Volver el sensor al frente (90 grados)
    myServo.write(90);
    delay(300);

    // Decidir el giro en base al espacio disponible
    if (distanceLeft > distanceRight) {
      girarIzquierda();
      delay(600); // Tiempo que dura el giro
    } else {
      girarDerecha();
      delay(600);
    }

    detener();
  }

  delay(100);
}
```

```
// --- Función para medir la distancia con el HC-SR04 ---
int medirDistancia() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH);
  int distance = duration * 0.034 / 2; // Fórmula para convertir tiempo en centímetros
  return distance;
}

// --- Funciones de Movimiento ---

void avanzar() {
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);
}

void retroceder() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  digitalWrite(in3, LOW);
  digitalWrite(in4, HIGH);
}

void girarIzquierda() {
```

```
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);
}

void girarDerecha() {
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, HIGH);
}

void detener() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);
}
```